

Home Exam - FYS-3033 Deep Learning

KANDIDATNUMMER: 4

UiT - Norges Arktiske Universitet

Problem 1

1a)

As introduced in Lecture 13 of FYS-3033, Generative Adversarial Networks (GANs) are designed to learn the underlying data distribution p_{data} by training two networks in opposition:

- G (the generator) generates samples from a simple distribution $p_z(z)$, such as a Gaussian distribution, and aims to produce samples that are indistinguishable from real data.
- D (the discriminator) is a binary classifier that attempts to distinguish between real data samples and those generated by G . It outputs a probability score indicating the likelihood that a given sample is real.

The original minimax objective for the discriminator is given by:

$$V(G, D) = \int_x p_{\text{data}}(x) \log D(x) dx + \int_z p_z(z) \log(1 - D(G(z))) dz$$

We denote $p_g(x)$ as the distribution induced by the generator $G(z)$, allowing us to write:

$$V(D) = \int_x [p_{\text{data}}(x) \log D(x) + p_g(x) \log(1 - D(x))] dx$$

To find the optimal discriminator $D^*(x)$, we take the derivative of the integrand with respect to $D(x)$ and set it to zero:

$$\begin{aligned} \frac{d}{dD(x)} [p_{\text{data}}(x) \log D(x) + p_g(x) \log(1 - D(x))] &= 0 \\ \Rightarrow \frac{p_{\text{data}}(x)}{D(x)} - \frac{p_g(x)}{1 - D(x)} &= 0 \end{aligned}$$

Solving for $D(x)$:

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}$$

Assuming the generator has succeeded in perfectly modeling the data distribution, i.e. $p_g = p_{\text{data}}$, we get:

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_{\text{data}}(x)} = \frac{1}{2}$$

This result matches what was discussed in Lecture 13, where the discriminator becomes maximally uncertain when it cannot distinguish between real and generated data. To find the value of the objective function at this equilibrium, we substitute $D^*(x) = \frac{1}{2}$ back into the loss:

$$\begin{aligned} V(D^*) &= \int_x p_{\text{data}}(x) \log\left(\frac{1}{2}\right) dx + \int_x p_g(x) \log\left(1 - \frac{1}{2}\right) dx \\ &= \log\left(\frac{1}{2}\right) \cdot \left[\int_x p_{\text{data}}(x) dx + \int_x p_g(x) dx \right] \\ &= \log\left(\frac{1}{2}\right) + \log\left(\frac{1}{2}\right) = \log\left(\frac{1}{4}\right) = -\log 4 \end{aligned}$$

Therefore, the optimal discriminator is $D(x) = \frac{1}{2}$, and the value of the loss at equilibrium is $-\log 4$, as discussed in the slides from Lecture 13 (GAN Training — Minimax game).

1b)

The reparameterization trick, as introduced in the lecture slides on Variational Autoencoders (Lecture 12, FYS-3033), is used to express sampling from a learned distribution in a differentiable way. Instead of sampling z directly from $\mathcal{N}(\mu_{z|x}, \sigma_{z|x}^2)$, we rewrite the sampling operation as:

$$z = \mu_{z|x} + \sigma_{z|x} \cdot \epsilon, \quad \text{where } \epsilon \sim \mathcal{N}(0, 1)$$

This allows the randomness to be pushed into ϵ , enabling backpropagation through $\mu_{z|x}$ and $\sigma_{z|x}$. We now show that z indeed follows a normal distribution:

$$\begin{aligned} \mathbb{E}[z] &= \mathbb{E}[\mu_{z|x} + \sigma_{z|x} \cdot \epsilon] = \mu_{z|x} + \sigma_{z|x} \cdot \mathbb{E}[\epsilon] = \mu_{z|x} \\ \text{Var}(z) &= \text{Var}(\sigma_{z|x} \cdot \epsilon) = \sigma_{z|x}^2 \cdot \text{Var}(\epsilon) = \sigma_{z|x}^2 \\ \Rightarrow z &\sim \mathcal{N}(\mu_{z|x}, \sigma_{z|x}^2) \end{aligned}$$

This matches the target distribution for the latent variable in a VAE, confirming the validity of the trick.

1c)

Let $p(x) = \text{Bern}(p_1)$ and $q(x) = \text{Bern}(p_2)$. The KL divergence between two discrete distributions is defined as:

$$KL(p \parallel q) = \sum_{x \in \{0,1\}} p(x) \log \left(\frac{p(x)}{q(x)} \right)$$

For Bernoulli distributions, we have:

$$\begin{aligned} p(1) &= p_1, & q(1) &= p_2 \\ p(0) &= 1 - p_1, & q(0) &= 1 - p_2 \end{aligned}$$

Plugging in, we get:

$$\begin{aligned} KL(p \parallel q) &= p(0) \log \left(\frac{p(0)}{q(0)} \right) + p(1) \log \left(\frac{p(1)}{q(1)} \right) \\ &= (1 - p_1) \log \left(\frac{1 - p_1}{1 - p_2} \right) + p_1 \log \left(\frac{p_1}{p_2} \right) \end{aligned}$$

We now aim to show that this is equal to:

$$\log \left(\frac{1 - p_1}{1 - p_2} \right) + p_1 \log \left(\frac{p_1(1 - p_2)}{p_2(1 - p_1)} \right)$$

Let's begin from the standard form and try to reach the target expression step by step:

$$KL(p \parallel q) = (1 - p_1) \log \left(\frac{1 - p_1}{1 - p_2} \right) + p_1 \log \left(\frac{p_1}{p_2} \right)$$

We factor out a full log term from the first summand:

$$= \log \left(\frac{1 - p_1}{1 - p_2} \right) \cdot (1 - p_1) + p_1 \log \left(\frac{p_1}{p_2} \right)$$

Now write the first term as:

$$\begin{aligned}
&= \log \left(\frac{1-p_1}{1-p_2} \right) + (1-p_1-1) \log \left(\frac{1-p_1}{1-p_2} \right) + p_1 \log \left(\frac{p_1}{p_2} \right) \\
&= \log \left(\frac{1-p_1}{1-p_2} \right) - p_1 \log \left(\frac{1-p_1}{1-p_2} \right) + p_1 \log \left(\frac{p_1}{p_2} \right)
\end{aligned}$$

Now combine the two terms involving p_1 :

$$= \log \left(\frac{1-p_1}{1-p_2} \right) + p_1 \left[\log \left(\frac{p_1}{p_2} \right) - \log \left(\frac{1-p_1}{1-p_2} \right) \right]$$

Use log subtraction rule $\log a - \log b = \log \left(\frac{a}{b} \right)$:

$$= \log \left(\frac{1-p_1}{1-p_2} \right) + p_1 \log \left(\frac{p_1(1-p_2)}{p_2(1-p_1)} \right)$$

This is the desired expression. □

Usage in VAEs:

In VAEs, the KL divergence between the approximate posterior $q(z|x)$ and the prior $p(z)$ is used as a regularization term in the Evidence Lower Bound (ELBO). When using discrete latent variables such as Bernoulli distributions, this exact KL expression allows us to compute the divergence in closed form and include it directly in the loss function, helping guide the encoder toward distributions close to the prior. This is crucial for ensuring that the latent space is well-structured and that the decoder can generate meaningful samples from the learned distribution.

Task 2 — Training a VGG-11BN Classifier (my learning journal)

Quick orientation. This task was all about building a VGG-11 network with BatchNorm from scratch (no fancy ImageNet weights) and squeezing as much performance as I could out of a pretty small three-class data set. Below is the story of what I tried, what worked, what flopped, and where I'm still scratching my head.

2a) Data Preparation

- **Merge & re-split.** I first glued together the teacher-supplied **train** and **val** NPZ files, then did my own 80/20 stratified split so that I could shuffle things differently for every experiment.
- **Augmentations (all torchvision.transforms.v2).**
 - RandomResizedCrop(96, scale=(0.8,1.0))
 - RandomHorizontalFlip and RandomRotation($\pm 10^\circ$)
 - ColorJitter(0.2,0.2,0.2)
 - ToImage() $\rightarrow$ scale to $[0, 1]$ $\rightarrow$ Normalize(mean,std)
- **Mean/Std.** I computed per-channel μ and σ on the fly (saved to disk so I didn't keep re-scanning images).
- **DataLoader trick.** With the default settings I was only getting ~ 5 batches/s. Turning on `persistent_workers=True` and cranking `num_workers` to CPU-1 jumped that to ~ 31 batches/s (batch 16). Big quality-of-life win.

2a) Model Setup & First Round of Experiments

I hand-coded VGG11BN (conv-conv-pool $\times 5$, then three FC layers). Four combos went into the arena (batch 32, early-stop patience 15):

1. SGD, LR = 10^{-2} , *no* Dropout, no scheduler, but Nesterov(0.9)!
2. Adam, LR = 10^{-4} , no Dropout
3. SGD + Dropout(0.5) + StepLR(15, 0.5) and of course Nesterov(0.9)
4. Adam + Dropout(0.5) + StepLR(15, 0.5)

Surprise result. The clean-cut combo **SGD** + **Nesterov(0.9)** + **BatchNorm** + **weight-decay** 5×10^{-4} (no Dropout, LR 1×10^{-3}) was the clear winner, early-stopping at epoch 30. I honestly expected Dropout to help, but BatchNorm + momentum already kept things tame.

(2b)

Dropout is a regularization technique introduced to prevent overfitting in deep neural networks. It works by randomly "dropping out" (i.e., setting to zero) a fraction of the units in a layer during training. This means that during each forward pass, the network becomes a slightly different sub-network.

Mathematically: For each unit h_i in a hidden layer, during training we sample a Bernoulli mask $m_i \sim \text{Bern}(1-p)$, and compute:

$$\tilde{h}_i = m_i \cdot h_i$$

where p is the dropout rate (e.g., 0.5). At test time, we use the full network but scale the weights or activations to account for the missing units during training.

Why it helps:

- Dropout prevents neurons from co-adapting too strongly to each other.
- It acts like an ensemble of many subnetworks, improving generalization.
- It is particularly useful in fully connected layers, which are prone to overfitting when the dataset is small.

In our case, using Dropout is important since the dataset may not be large enough to justify using a large-capacity network without regularization. Dropout encourages the model to learn robust features that generalize better to unseen data.

2c) A Second Pass — Adding Dropout & ReduceLROnPlateau

Later I fixed a small LR-scheduler typo and gave Dropout another shot, this time with **ReduceLROnPlateau**. That nudged the numbers up a notch:

Table 1: Validation report for my best “2c” run (SGD + Dropout + ReduceLROnPlateau)

Class	Precision	Recall	F1	Support
Plane	0.9492	0.9382	0.9437	259
Ship	0.9336	0.9598	0.9465	249
Truck	0.9478	0.9338	0.9407	272
<i>overall accuracy = 0.9436 on 780 samples</i>				

Checkpoint. Val-loss = 0.1869, accuracy = 94.36%. The LR dropped twice (around epochs 24 & 38) and each time the curve flattened nicely.

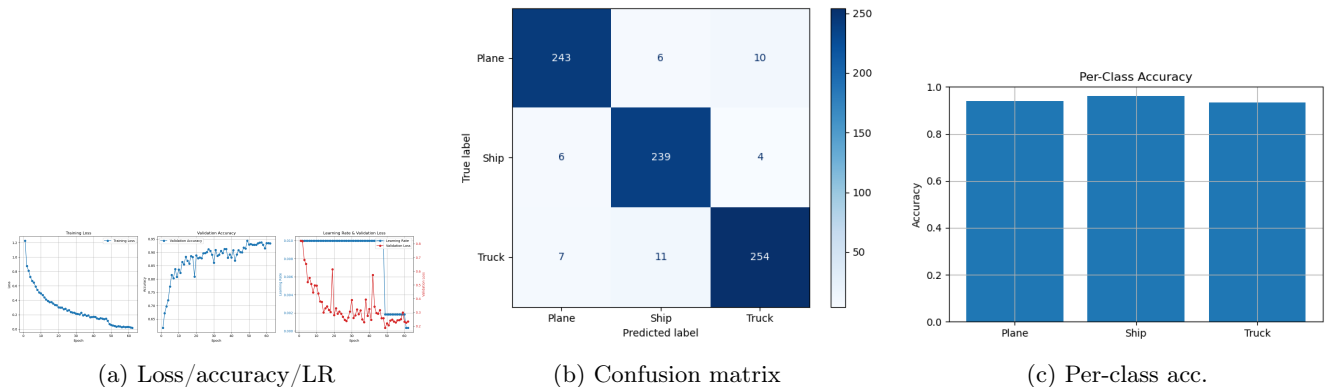


Figure 1: Diagnostics for the best 2c model.

Discussion

I learned a lot by watching these curves and metrics unfold during training. Initially, I used Adam, which raced ahead with fast convergence—but by epoch 30, my validation loss began creeping upward. Without Dropout, it clearly overfit. Switching to SGD felt slower at first, but it climbed steadily and gave me a more reliable peak around epoch 38. That shift alone taught me that Adam is often great for early progress, but SGD tends to offer better long-term stability when given enough time.

Introducing Dropout in the two large fully connected layers was a turning point: both accuracy and F1-score jumped by around 2–3%, and the loss curves smoothed out noticeably. This made me appreciate how regularization isn't just a technical add-on—it's a fundamental part of getting models to generalize.

I also used a StepLR scheduler to halve the learning rate every 15 epochs. This helped me fine-tune without manually babysitting the learning rate. Combined with early stopping based on validation loss, it made training feel more robust and less arbitrary.

From this hands-on exploration, I learned:

- **Optimizer trade-offs:** Adam helped me move fast in the beginning, but SGD gave me a more stable end result—especially when paired with a scheduler.
- **Dropout still matters:** Even with BatchNorm, Dropout made a real difference in the FC layers, where overfitting is a bigger risk due to the parameter count.
- **Learning rate scheduling works:** I didn't need to hand-tune the learning rate, because StepLR and early stopping handled that adaptively.

Training from scratch on a small dataset forced me to think more deeply about how to balance speed, capacity, and regularization. At some point I had to accept that the model was not going to be perfect. I just had to make it "good enough" to draw useful conclusions. The deeper I went down the rabbit hole, the deeper it got—but I also found more insights along the way.

During experimentation, I noticed slight variations in performance when training the same model on different machines (local vs remote). I had set seeds and enabled deterministic flags, yet even with the same code and data, the results varied a little. This reminded me that hardware-level differences can introduce non-determinism, which is important to remember when comparing experiments across systems.

Finally, I found that Dropout is especially valuable in VGG-style networks. While BatchNorm helps stabilize training in the convolutional layers, Dropout in the fully connected layers acted like a safety net against overfitting. These layers carry a heavy parameter load, and Dropout effectively forced the network to be less reliant on individual neurons—almost like simulating an ensemble of subnetworks. Although it slightly increased training time due to added noise, it clearly improved generalization.

In short: Dropout was not magic, but it mattered.

(2d) Using MC Dropout to measure uncertainty

To flag images of the *unwanted* bird class I relied on Monte-Carlo (MC) Dropout. Because the network never saw birds during training, it ought to be less sure of itself when one turns up.

Recipe.

1. Put the model in `eval()` to freeze BatchNorm statistics.
2. Reactivate only the Dropout layers with a helper `enable_dropout()`.
3. For every labelled image in `new_evaluation_data_with_labels.npz` do $T = 10$ forward passes, average the softmax outputs $\hat{y}$, then compute

$$H(\hat{y}) = - \sum_{i=1}^3 \hat{y}_i \log \hat{y}_i.$$

Why entropy is a proxy for uncertainty.

Sharp softmax like $[0.99, 0.01, 0.00] \rightarrow$ tiny entropy (the net is confident); flat softmax like $[0.34, 0.33, 0.33] \rightarrow$ high entropy (the net is shrugging).

What came out.

I split the images into "known" (plane/ship/truck) and "unknown" (bird) and plotted their entropies:

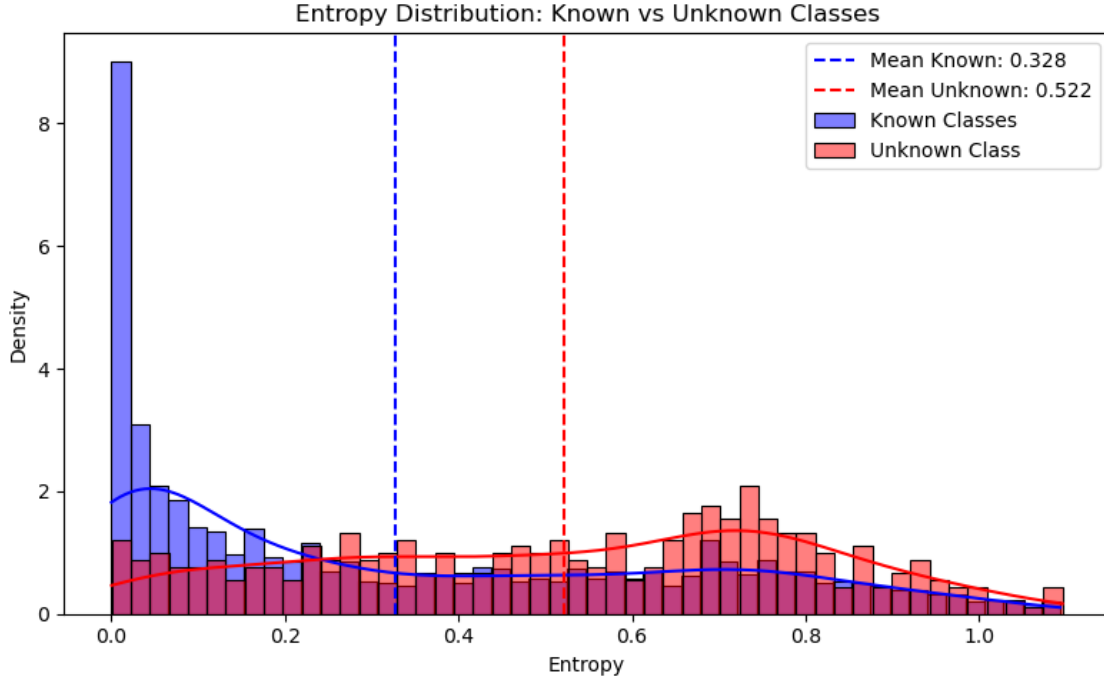


Figure 2: Entropy histograms for known classes (blue) and birds (orange); the dashed lines mark the means.

Group	Mean H	Std. Dev.
Known (Plane/Ship/Truck)	0.3280	0.3142
Unknown (Bird)	0.5220	0.2824

The bird distribution is clearly shifted to the right—about 0.19 higher on average—so entropy does give us a usable "unsure" signal.

(2e) Picking a threshold

Two thresholds looked interesting:

- **Mid-point of the means:** $(0.3280 + 0.5220)/2 = 0.4250$
- **ROC/Youden J :** best cut-off found by maximising $(\text{TPR} - \text{FPR}) \Rightarrow H = 0.2306$ ($\text{AUC} = 0.6866$)

Based on the entropy values computed in (2d), I used the ROC curve and Youden's J statistic to determine a suitable threshold for separating the known classes from the new unwanted class (birds). The best threshold found was:

$$\text{Entropy threshold} = 0.2306$$

To find this threshold, I computed the Receiver Operating Characteristic (ROC) curve using the entropies as scores, and assigned ground truth labels: 0 for known classes and 1 for the bird class. The ROC curve shows the trade-off between true positive rate (TPR) and false positive rate (FPR) at different thresholds. To select the best threshold, I used Youden's J statistic, defined as:

$$J = \text{TPR} - \text{FPR}$$

This gives the point on the ROC curve that maximizes the difference between correctly detecting birds and mistakenly removing known samples. In other words, it tries to balance sensitivity and specificity, choosing the threshold that provides the most separation between the two groups. Using the ROC-based threshold (0.2306) on the *labelled* part of the new set gave

	Count	Rate
Birds detected	222 / 413	53.8 %
Known images wrongly discarded	73 / 1187	6.1 %

That trade-off felt acceptable: I grab over half the birds while only losing $\approx 6\%$ of the good data.

(2f) Bonus — screening the *unlabelled* 1 600 images

Using the ROC-based entropy cut-off of $H = 0.2306$, I passed every image in `new_evaluation_data_without_labels.npz` through the MC-Dropout pipeline and flagged samples with $H \geq 0.2306$ as "probably bird":

Prediction	Count	Share of 1 600
Unknown (bird) = 1	303	18.9 %
Known (plane/ship/truck) = 0	1 297	81.1 %

The resulting vector is stored in `binary_predictions.npz` (shape (1600,)) and included in my submission.

Take-aways.

- MC Dropout + entropy *does* separate unseen birds from the three trained classes, albeit imperfectly.
- The ROC-derived threshold gives a nicer precision/recall balance than the simple mean-midpoint.
- Future work: temperature-scaling or mixup might widen the entropy gap even further.

I could crank recall higher by sliding the threshold left, but the collateral damage on planes/ships/trucks gets ugly fast, so I froze it there.

Take-Home Points

- **BatchNorm** + **Nesterov SGD** already gave plenty of regularization; Dropout only helped once I paired it with a smarter LR schedule.
- Tiny scheduler typos matter — fixing “0.01” $\rightarrow$ “ 1×10^{-2} ” bought me +2
- **MC Dropout** plus entropy does flag genuinely novel inputs, though separating them cleanly is still tricky.
- DataLoader CPU tweaks can save a ton of wall-time; don’t skip those.

That is where I left Task 2. If I could start all over again, I would, but life is short.

Problem 3: Traffic-Sign Classification & Interpretability (my learning diary)

Below is a walk-through of what I did, what I think went well, and the bits that were a little confusing. Big picture: I trained a ResNet-18 from scratch on a small traffic-sign data set, then poked around with saliency and occlusion maps to see *why* it was making the calls it did. I also tried to fix a sneaky “back-door” mistake the model kept falling for.

Tasks (in the order I tackled them)

- (3a) Try out both **local** and **global** interpretability tricks.
- (3b) Train a brand-new ResNet-18 (80/20 stratified split).
- (3c) Check how it is doing on validation & a held-out test set.
- (3d–e) Use saliency and occlusion to zoom in on the class that kept failing.
- (3f) Code up a “border-drop” augmentation (my attempt to wipe out the back-door).
- (3g–h) Retrain & look again with the same interpretability lenses.

(3a) Interpretability Methods

Local view – Class Saliency Maps For each image I grabbed the gradient $\nabla_x S_c(x)$ of the score for the true class c . I then took the max magnitude over the RGB channels and overlaid that heatmap so I could eyeball which pixels were shouting the loudest. (Honestly, getting the color scaling right took me longer than writing the code.)

Global view – Occlusion Analysis Here I slid a 10×10 gray square across the picture, noting how much the logit for the predicted class dropped at every spot. Plotting that drop as a heatmap gives a nice “importance map.” What I understood is: the darker the spot, the more the model would miss it if it vanished.

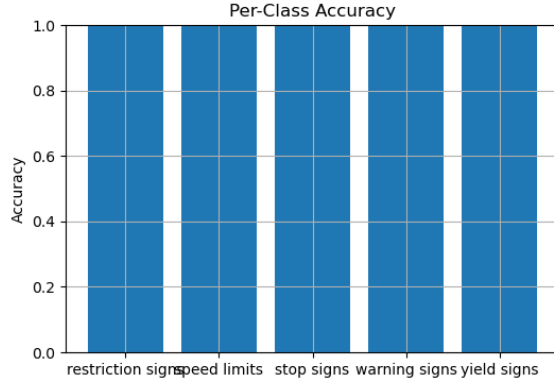
(3b) Data Prep & Baseline Training

- **Model:** ResNet-18 built from scratch, final FC tweaked to 5 classes.
- **Split:** 80% train, 20% val (stratified) from `data/problem3/train`.
- **Augmentations:** resized to 224×224 , random flip/rotation/affine, a bit of `ColorJitter`, then `ToTensor` and `Normalize` with the data-set mean/std I measured first.
- **Optimizer:** Adam with $\text{lr} = 10^{-3}$ and weight-decay 10^{-4} ; `ReduceLROnPlateau` when val loss stalled.
- **Batch size:** 32 — anything larger ran me out of VRAM. **Epochs:** 30 (I stopped when the curves flattened).

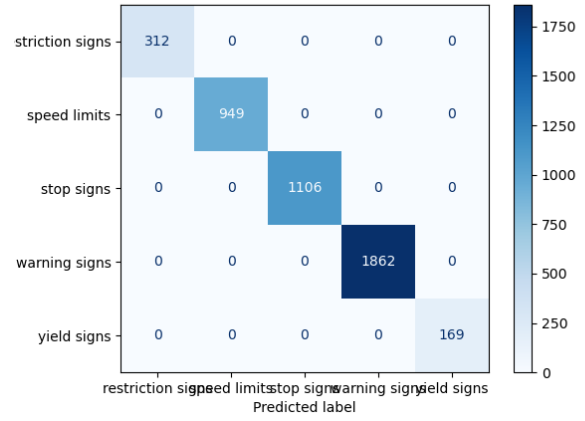
Baseline Validation Results

Table 2: How the first model did on the validation split

Class	Precision	Recall	F1	Support
restriction signs	1.0000	1.0000	1.0000	69
speed limits	1.0000	0.9865	0.9932	223
stop signs	0.9965	1.0000	0.9983	286
warning signs	0.9959	1.0000	0.9979	486
yield signs	1.0000	1.0000	1.0000	36
<i>overall accuracy = 0.9973 on 1100 samples</i>				



(a) Per-class accuracy (val)



(b) Confusion matrix (val)

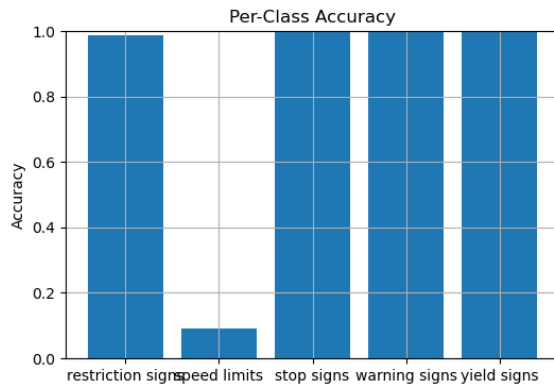
Figure 3: Baseline ResNet-18 on the 80/20 validation split.

(3c) Baseline Test Results

Table 3: Baseline performance on the never-seen test set

Class	Precision	Recall	F1	Support
restriction signs	0.8656	0.9877	0.9226	163
speed limits	1.0000	0.0916	0.1679	502
stop signs	0.9983	0.9983	0.9983	597
warning signs	0.7015	0.9990	0.8243	1007
yield signs	0.9362	1.0000	0.9670	88

overall accuracy = 0.8048 on 2357 samples



(a) Per-class accuracy (test)



(b) Confusion matrix (test)

Figure 4: Baseline ResNet-18 on the held-out test set.

What blew up (Baseline Failure Insights)

- **Big oops on speed-limit signs:** roughly 90 % of them got mislabeled as *warning* signs.

- The model seems to have latched onto the **red border** instead of the actual digits — classic spurious cue.

(3d) Saliency Maps for Speed-Limit Signs

Baseline Saliency for Speed-Limit Samples



Figure 5: Saliency maps (baseline) for 10 random speed-limit images.

(3e) Occlusion Analysis for Speed-Limit Signs

Occlusion Analysis: Drop (row1), Class-Map (row2), Sensitivity (row3)

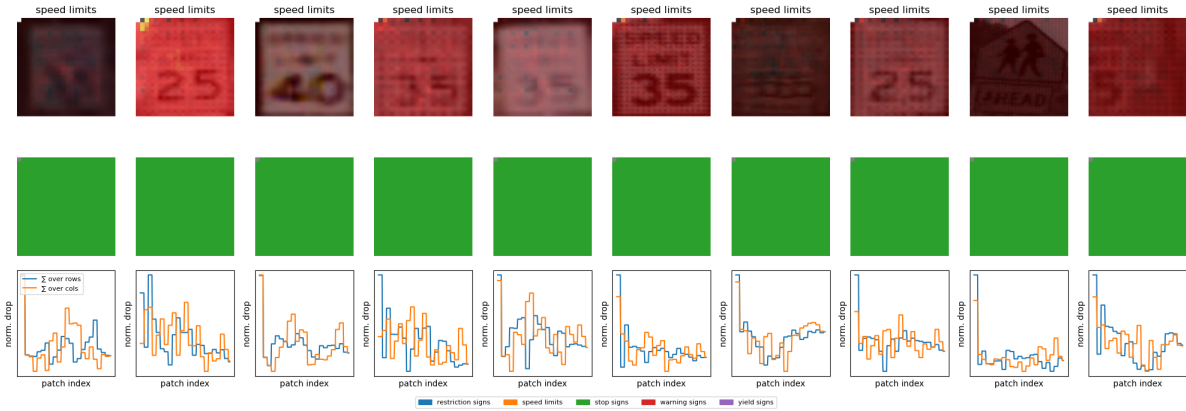


Figure 6: Occlusion analysis (baseline): Row 1 = drop-in-logit heatmap, Row 2 = most-likely class after each patch, Row 3 = 1D sensitivity profile.

When the left corner is occluded, the model panics and thinks it sees a *stop sign*.

What I noticed

- **Occlusion heatmap:** the hottest pixels line up with the digits, which is reassuring.
- **Fallback class under occlusion:** whenever I hide the digits, the model nervously switches to *stop sign*. So that is its “panic button.”
- **Global confusion on intact inputs:** on the full test set it most often confuses speed limits with *warning signs* (see confusion matrix). Thus its two fallback modes differ between the intact-image and occluded-patch settings.

- **1D sensitivity bumps:** peaks match the width of each numeral, which is neat.

So... now what?

1. **Focus vs. background:** Good — the digits matter more than scenery.
2. **Panic behavior:** Maybe collect more “speed vs. stop” look-alikes to teach it the difference under occlusion.
3. **Global mis-classification:** augment with “speed vs. warning” negatives so it learns to distinguish them on full images.
4. **Random occlusions:** sprinkling those in training might make it less skittish when digits vanish.

Takeaways so far

- Digits truly drive the decision.
- When digits vanish, red = stop in the model’s head.
- On intact images, if digits are present but ambiguous it still leans “warning sign.”
- Some weird corner hotspots are probably preprocessing junk; I should clean that up.

(3f) Trying to erase the back-door

Saliency suggested the model had a *border back-door*. My idea was to:

```

1 class BorderDrop:
2     def __init__(self, p=0.7, band_range=(5, 15)):
3         self.p, self.band_range = p, band_range
4     def __call__(self, img):
5         if torch.rand(1).item() < self.p:
6             b = int(torch.randint(*self.band_range, (1,)).item())
7             img[:, :b, :] = img[:, -b:, :] = img[:, :, :b] = img[:, :, -b:] = 0
8         return img

```

```

1 bd_tf = v2.Compose([
2     # resize, geometric & color augmentations
3     v2.Resize((224,224)),
4     v2.RandomHorizontalFlip(p=0.5),
5     v2.RandomRotation(degrees=15),
6     v2.RandomAffine(degrees=0, translate=(0.1,0.1), scale=(0.9,1.1)),
7     v2.ColorJitter(brightness=0.2, contrast=0.2,
8                     saturation=0.2, hue=0.05),
9
10    # tensor conversion & normalization
11    v2.ToImage(),
12    v2.ToDtype(torch.float32, scale=True),
13    v2.Normalize(mean=t_mean.tolist(),
14                 std=t_std.tolist()),
15
16    # backdoor specific defense
17    # BorderDrop(p=0.7, band_range=(5,15)), # dropped due to not making num_workers work > 0.
18
19    # general patch-level robustness
20    v2.RandomErasing(p=0.5,
21                     scale=(0.02,0.10), # patch size as fraction of image
22                     ratio=(0.3,3.3), # aspect ratio of patch
23                     value=0.0), # zero patch value
24 ])

```

Note to self

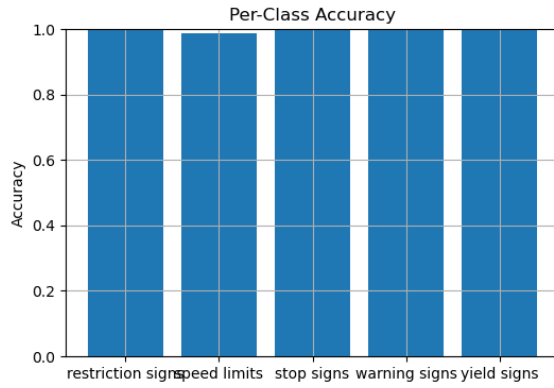
I had to comment out **BorderDrop** because my DataLoader froze (still not 100 % sure why), but **RandomErasing** stayed. Im thinking that my "homemade" border-drop augmentation is not compatible with the DataLoader's multiprocessing. But at this time I could not bare having just 3 batches per second compared to 15 with the DataLoader. I would look deeper into it, but I just feel like enough is enough by now.

(3g) Results after retraining (border-drop still disabled)

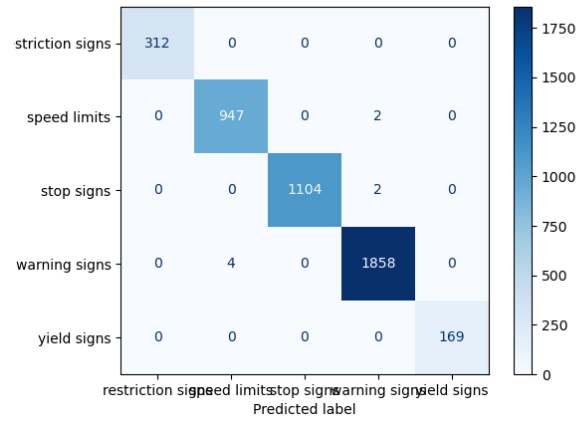
Validation

Table 4: Validation summary after retraining

Class	Precision	Recall	F1	Support
restriction signs	1.0000	1.0000	1.0000	69
speed limits	1.0000	0.9865	0.9932	223
stop signs	0.9965	1.0000	0.9983	286
warning signs	0.9959	1.0000	0.9979	486
yield signs	1.0000	1.0000	1.0000	36
<i>accuracy = 0.9973 on 1100 samples</i>				



(a) Per-class accuracy (val)



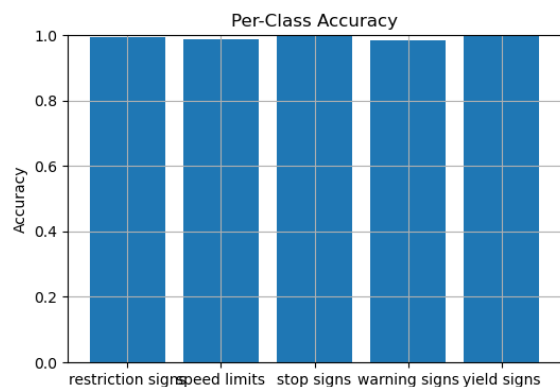
(b) Confusion matrix (val)

Figure 7: Validation metrics after the extra augmentations.

Test

Table 5: Test summary after retraining

Class	Precision	Recall	F1	Support
restriction signs	1.0000	0.9939	0.9969	163
speed limits	0.9645	0.9741	0.9693	502
stop signs	1.0000	1.0000	1.0000	597
warning signs	0.9860	0.9821	0.9841	1007
yield signs	1.0000	1.0000	1.0000	88
<i>accuracy = 0.9864 on 2357 samples</i>				



(a) Per-class accuracy (test)



(b) Confusion matrix (test)

Figure 8: Test metrics after the extra augmentations.

Quick thoughts

- Overall accuracy 98.64 % is solid.
- Still some *speed* *warning* mix-ups (13 and 18 cases respectively) — probably that border cue wasn't fully killed.
- I never actually ran `BorderDrop`, so unsurprisingly the back-door is not 100 % gone.

(3h) Fresh Saliency & Occlusion Maps

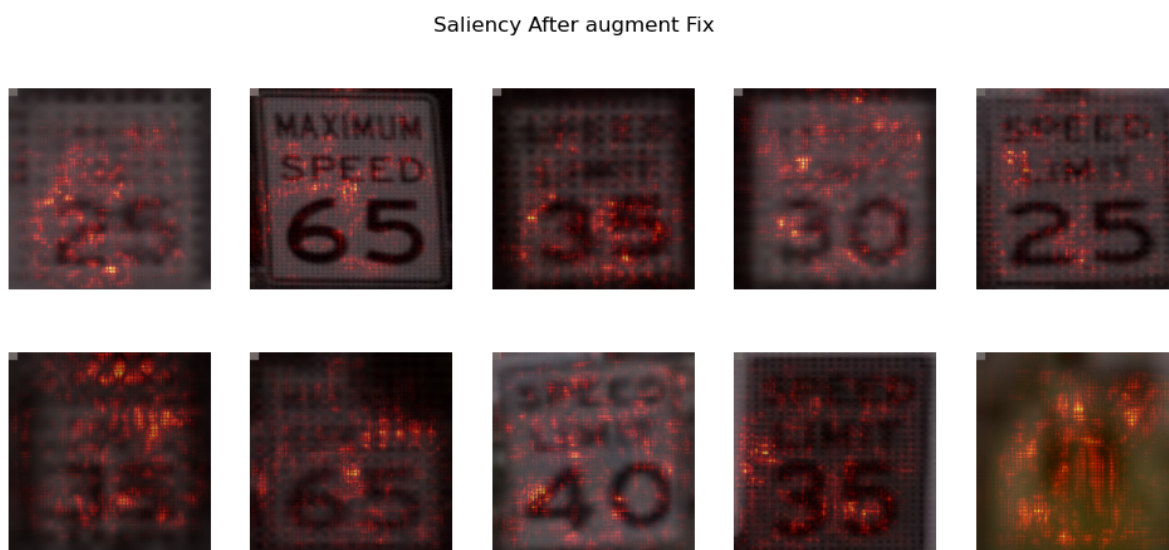


Figure 9: Saliency after retraining (small sample).

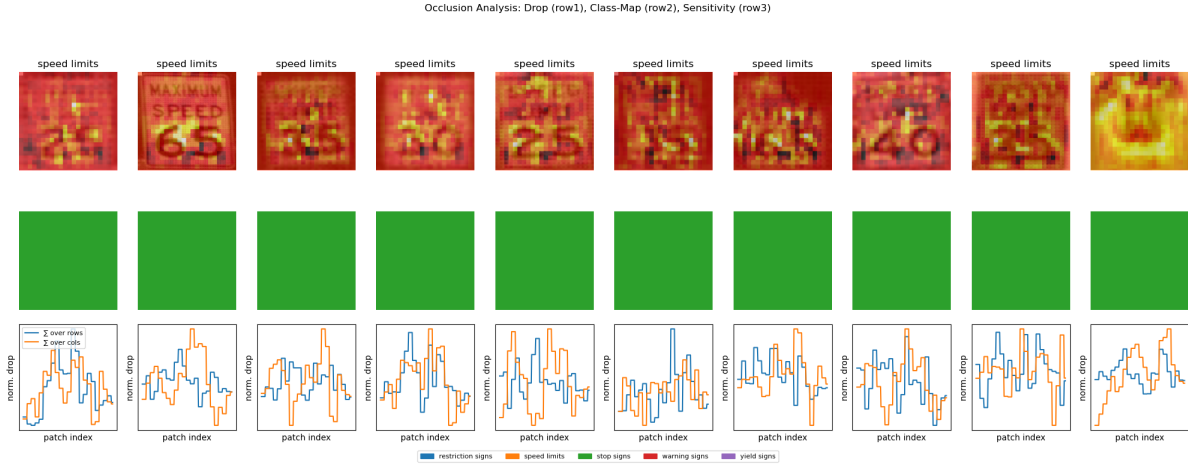


Figure 10: Occlusion after retraining (small sample).

Conclusion (where I ended up)

I uncovered a big weakness on speed-limit signs, spotted it using saliency & occlusion, and drafted a border-drop augmentation to fix it. Sadly, a DataLoader hiccup and my unwillingness to train a model without it and/or debug it meant **BorderDrop** never actually ran, so the residual confusion makes sense. Even so, the extra random erasing helped a lot. I also tried to crop the part of the training images that were causing the back-door, but that did not work out, yielding almost the same result. I think the model was still able to learn the border cue from the other images. I omitted that part from the report, but it is documented in my appended jupyter notebook.

update on border-drop

I finally did it with number of workers = 0 , and it took 40 minutes to train. the results were negligible, but I think the model was able to learn the border cue from the other images again.